



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA
Escuela Profesional de Ingeniería de Sistemas

INFORME DE LABORATORIO
“Videojuego tipo Snake

Curso: Programación II

Docente: Breno Solim Luque Zuñiga

Jiménez Romero, Josué Andre (2022074259)

Tacna – Perú
2026



INDICE

Contenido

INDICE.....	2
I. INFORMACIÓN GENERAL	3
- Objetivos	3
- Equipos, materiales, programas y recursos utilizados	3
II. MARCO TEORICO.....	3
III. PROCEDIMIENTO	4
IV. ANALISIS E INTERPRETACION DE RESULTADOS	14
CONCLUSIONES	14
RECOMENDACIONES.....	14



INFORME DE LABORATORIO

TEMA: Videojuego Tipo Snake

I. INFORMACIÓN GENERAL

- **Objetivos:** Comprender los fundamentos de la programación orientada a objetos (POO), el manejo de eventos y la renderización de gráficos 2D en C# a través del desarrollo de un videojuego interactivo clásico (Snake) utilizando Windows Forms.
- **Equipos, materiales, programas y recursos utilizados:**
 - Azure Data Studio
 - Base de datos: TSQL.bak
 - Al menos 4GB de RAM.
 - Librerías gráficas: System.Drawing

II. MARCO TEORICO

El laboratorio se estructuró en cuatro partes que reflejan la arquitectura básica de un videojuego 2D:

- **Parte I: Programación Orientada a Objetos (POO)**

Se crearon entidades independientes (Serpiente, Comida, Segmento) para separar la lógica del juego. Esto permite el encapsulamiento de datos (como las coordenadas y el tamaño) y facilita el mantenimiento del código.

- **Parte II: Bucle de Juego (Game Loop) y Eventos**

Se implementó un bucle central controlado por un componente Timer (gameTimer), el cual dicta el ritmo de actualización lógica (ticks). Además, se usaron eventos de teclado (KeyDown) para capturar asincrónicamente las interacciones del usuario de manera fluida.



- **Parte III: Gráficos y Renderizado (GDI+)**

Se utilizó el evento Paint del panel de dibujo junto con la clase Graphics para dibujar en pantalla figuras primitivas (rectángulos para la serpiente y elipses para la comida). Se activó la propiedad *DoubleBuffered* mediante reflexión (Reflection) para evitar el parpadeo visual (*flickering*) durante la actualización de fotogramas.

- **Parte IV: Detección de Colisiones y Lógica de Estado**

Se implementaron algoritmos para detectar intersecciones de coordenadas (cabeza de serpiente vs. comida, bordes de la pantalla o su propio cuerpo) para gestionar estados críticos del juego como "Pausado", "Ganar puntos" o "Game Over".

III. PROCEDIMIENTO

Control	Propiedad	Valor
Form1	Text, BackColor, FormBorderStyle	"Juego de Snake", Black, FixedSingle
pnlTablero	BackColor, Size, BorderStyle	Navy, 600; 500, FixedSingle
pnlInfo	BackColor, Size	Black, 180; 500
lblTitulo	Text, Font, ForeColor	"Juego de Snake", Courier New 10 Negrita, Yellow
lblInstrTitulo	Text, Font, ForeColor	"Instrucciones:", Courier New 8 Negrita, White
lblInstruccion	Text, Font, ForeColor	"↑ = Arriba ↓ = Abajo → = Der.\n← = Izq. P = Pausa Esc = Game Over", Courier New 8, Yellow



INFORME DE LABORATORIO– Videojuego tipo Snake

lblCreadoTit	Text, Font, ForeColor	"Creado por:", Courier New 8 Negrita, White
lblAutor	Text, Font, ForeColor	"Josué Jiménez", Courier New 8, Cyan
lblPuntajeTit	Text, Font, ForeColor	"Puntaje:", Courier New 8 Negrita, White
lblPuntaje	Text, Font, ForeColor	"Pts: 0", Courier New 12 Negrita, Yellow
btnReiniciar	Text, Font, BackColor, ForeColor	"Reiniciar", Courier New 9 Negrita, DarkGreen, White
gameTimer	Interval	150

Figura 1

Modelo del formulario SNAKE

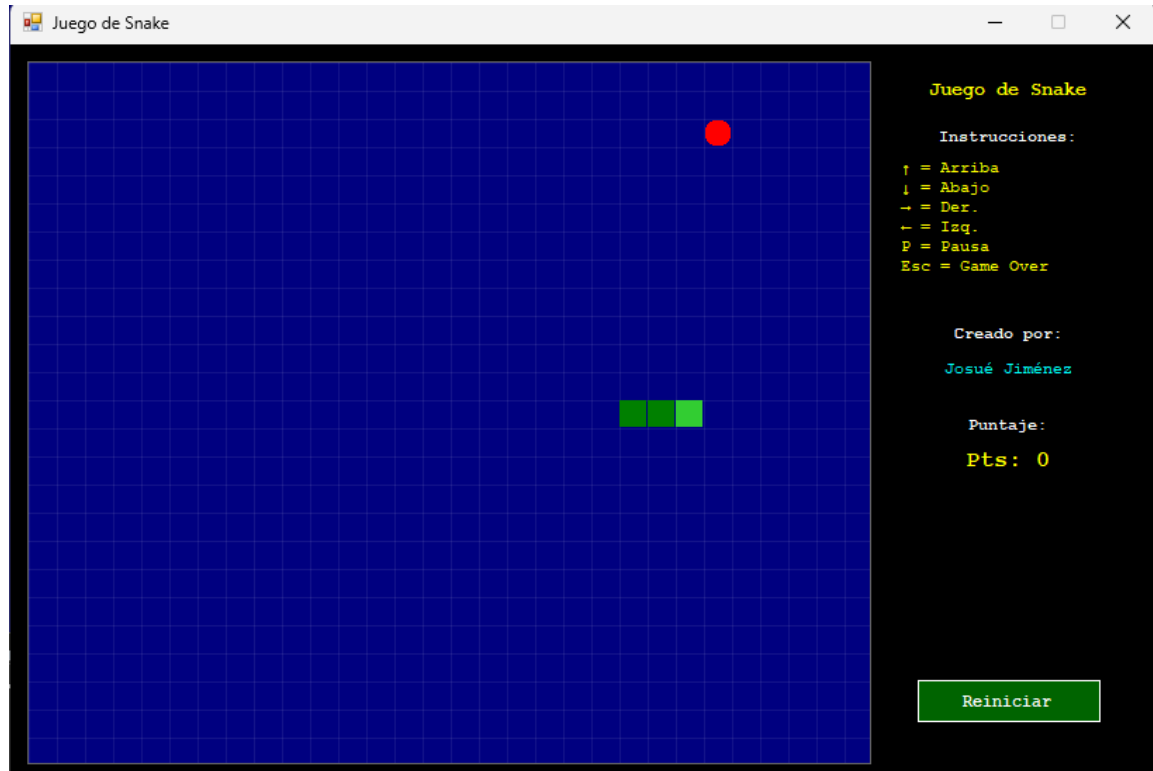




Figura 2

Impresión en el panel cuando el jugador pierde

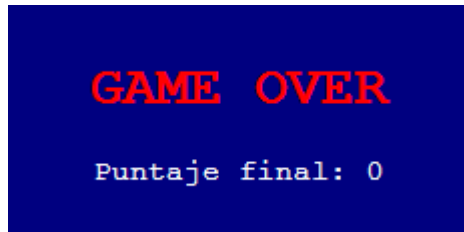


Figura 3

Variables constantes y de estado

```
//Constantes de tablero:
private const int TAM_CELDA = 20;           // px por celda
private const int COLS = 30;               // 600 / 20
private const int FILAS = 25;              // 500 / 20
private const int PTS_COMIDA = 10;

//Estado de juego:
private Serpiente _serpiente;
private Comida _comida;
private int _puntaje;
private bool _pausado;
private bool _gameOver;
```



Figura 4

Constructor

```
//Constructor
1 referencia
public Form1()
{
    InitializeComponent();
    pnlTablero.GetType()
        .GetProperty("DoubleBuffered",
            System.Reflection.BindingFlags.Instance |
            System.Reflection.BindingFlags.NonPublic)
        ?.SetValue(pnlTablero, true, null);

    // Eventos
    IniciarJuego();
    gameTimer.Start();
    this.Focus();
}
```

Figura 5

Función IniciarJuego()

```
private void IniciarJuego()
{
    _serpiente = new Serpiente(TAM_CELDA, COLS / 2, FILAS / 2);
    _comida = new Comida(TAM_CELDA, COLS, FILAS);
    _puntaje = 0;
    _pausado = false;
    _gameOver = false;
    lblPuntaje.Text = "Pts: 0";
}
```

Nota: Se instancia a la clase Serpiente e inicializa las configuraciones de arranque para cada partida



Figura 6

Configuración de Acción para el Timer

```
1 referencia
private void gameTimer_Tick(object sender, EventArgs e)
{
    if (_pausado || _gameOver) return;

    bool comio = _serpiente.Mover(_comida, COLS, FILAS);

    if (!_serpiente.Viva)
    {
        _gameOver = true;
        gameTimer.Stop();
    }
    else if (comio)
    {
        _puntaje += PTS_COMIDA;
        lblPuntaje.Text = $"Pts: {_puntaje}";
        _comida.Reposicionar(COLS, FILAS);

        // Acelera un poco cada 50 pts
        if (_puntaje % 50 == 0 && gameTimer.Interval > 60)
            gameTimer.Interval -= 10;
    }

    pnlTablero.Invalidate();
}
}
```

Figura 7

Configuración de Acción para el formulario

```
private void Form1_KeyDown(object sender, KeyEventArgs e)
{
    switch (e.KeyCode)
    {
        case Keys.Up:
            if (_serpiente.Direccion != Direccion.Abajo)
                _serpiente.Direccion = Direccion.Arriba;
            break;
        case Keys.Down:
            if (_serpiente.Direccion != Direccion.Arriba)
                _serpiente.Direccion = Direccion.Abajo;
            break;
        case Keys.Left:
            if (_serpiente.Direccion != Direccion.Derecha)
                _serpiente.Direccion = Direccion.Izquierda;
            break;
        case Keys.Right:
            if (_serpiente.Direccion != Direccion.Izquierda)
                _serpiente.Direccion = Direccion.Derecha;
            break;
        case Keys.P:
            _pausado = !_pausado;
            if (!_pausado) gameTimer.Start();
            pnlTablero.Invalidate();
            break;
        case Keys.Escape:
            _gameOver = true;
            gameTimer.Stop();
            pnlTablero.Invalidate();
            break;
    }
}
}
```




Figura 8

Configuración del botón Reiniciar

```
1 referencia
private void btnReiniciar_Click(object sender, EventArgs e)
{
    gameTimer.Interval = 150; // resetea velocidad inicial
    IniciarJuego();
    gameTimer.Start();
    pnlTablero.Invalidate();
    this.Focus();
}
```

IV. ANALISIS E INTERPRETACION DE RESULTADOS

Al ejecutar se observó que el entorno gráfico responde correctamente al bucle del juego. La renderización es fluida gracias a la activación de *DoubleBuffered*. Las validaciones de coordenadas son precisas: al tocar la pared o chocar consigo misma, el juego detecta correctamente la pérdida cambiando la bandera booleana *_gameOver* y deteniendo el temporizador. La dificultad se ajusta de manera imperceptible pero efectiva, acelerando los FPS (reduciendo el Interval de 150ms hacia abajo) cada 50 puntos acumulados, lo cual demuestra que la lógica matemática del flujo del juego y el renderizado están bien sincronizados.

V. CONCLUSIONES

- Se logró desarrollar un videojuego funcional que implementa y coordina clases en C# para manejar estados visuales y lógicos en tiempo real.
- Se identificó la importancia crítica de utilizar listas dinámicas (*List<T>*) para manejar estructuras que cambian de tamaño constantemente, como el cuerpo de la serpiente, resultando más eficientes y fáciles de manipular que los arreglos estáticos tradicionales.



INFORME DE LABORATORIO– Videojuego tipo Snake

- Se comprendió cómo el manejo adecuado de eventos (teclado y reloj temporal) y la invalidación estratégica de controles (Invalidate()) son fundamentales para la creación de interfaces interactivas fluidas.

VI. RECOMENDACIONES

- Mejorar la arquitectura aplicando un patrón de diseño como MVC (Modelo-Vista-Controlador) para desacoplar completamente la lógica del juego de los controles del formulario de Windows.
- Implementar un sistema de almacenamiento local (ej. archivos .json o .txt) para guardar un registro de la puntuación máxima histórica (*High Score*).
- Añadir recursos multimedia, como efectos de sonido al comer o al perder, y reemplazar las figuras geométricas pintadas (SolidBrush) por *sprites* (imágenes 2D) para darle un aspecto visual más pulido.